

Amazon Clone 22 — Technical Architecture & Documentation Manual

Scope note: This manual documents the architecture we actually built together: Angular 22 standalone components, feature-based organization, Angular Signals for client state, HttpClient, json-server, authentication guards, product browsing/details, cart, checkout, and order completion. Where the conversation did not establish an implementation—such as custom pipes, custom directives, NgRx, OnPush, auth-token interceptors, resolvers, or CanDeactivate guards—I explicitly document that they are not part of the current architecture rather than inventing code.

1. High-Level Architectural Overview

1.1 Architectural Pattern

The application is a client-side monolithic Angular Single-Page Application with feature-based organization and service-oriented state management.

The architecture combines four important patterns.

Feature-based architecture organizes business capabilities into auth, home, products, cart, and checkout.

Container/presentation separation appears most clearly in the home feature:

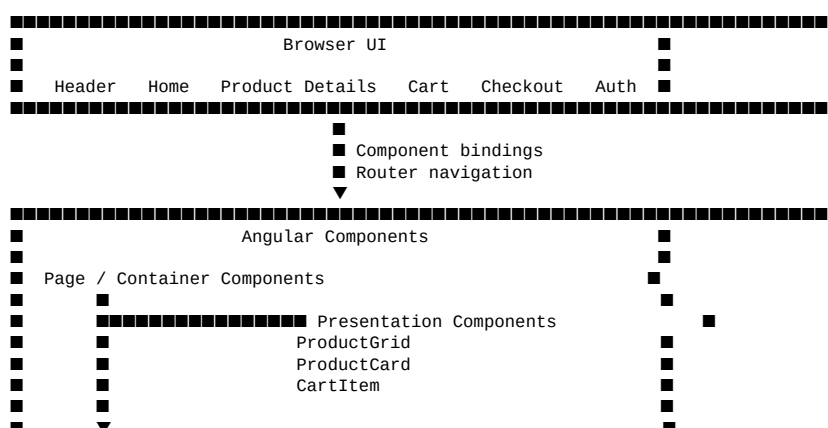
- Home acts as the page/container.
- ProductGrid controls product collection rendering.
- ProductCard renders one individual product.
- Banner, HeroSlider, and ShoppingCards handle isolated UI responsibilities.

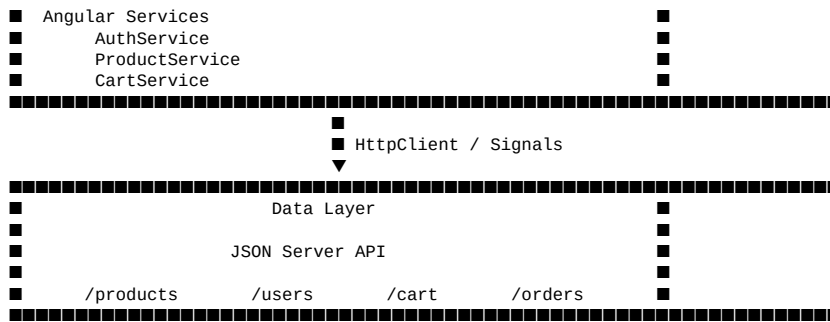
Singleton service architecture centralizes shared business logic:

- AuthService
- ProductService
- CartService

Signal-based reactive state management keeps shared client state synchronized with templates without introducing NgRx or another external state-management framework.

The logical architecture is:





1.2 Module Strategy

The project uses the modern Angular Standalone Component architecture.

It does not use the traditional root AppModule.

The application is bootstrapped through:

```

main.ts
├─
└─ bootstrapApplication()
    ├─ App
    └─ appConfig
        ├─ Router
        ├─ HttpClient
        └─ Angular providers

```

This architecture is appropriate for Angular 22 because dependencies are imported directly by standalone components.

For example:

```

@Component({
  selector: 'app-product-grid',
  standalone: true,
  imports: [
    ProductCard
  ],
  templateUrl: './product-grid.html',
  styleUrls: ['./product-grid.css']
})
export class ProductGrid {
  products = input.required<Product[]>();
}

```

The component explicitly declares its dependency on ProductCard.

There is no feature NgModule responsible for declaring both components.

1.3 Directory Structure

The final project evolved into the following feature-oriented architecture:

```

amazon-clone-22/
├─ public/
│ └─ assets/
│     └─ DATA/
│         └─ products.json
│     └─ images/
│         └─ shopping/

```

```

■
■
■ checkoutAd.png
■ hero-1.png
■ hero-2.png
■ hero-3.png
■ login-banner.png
■ prime.png
■
■ src/
■   app/
■     core/
■       api/
■         api.config.ts
■       data/
■       guards/
■         auth-guard.spec.ts
■         auth-guard.ts
■       interceptors/
■       services/
■         auth.spec.ts
■         auth.ts
■         cart.spec.ts
■         cart.ts
■         product.spec.ts
■         product.ts
■     features/
■       auth/
■         pages/
■           login/
■             login.css
■             login.html
■             login.spec.ts
■             login.ts
■           register/
■             register.css
■             register.html
■             register.spec.ts
■             register.ts
■       cart/
■         components/
■           cart-item/
■             cart-item.css
■             cart-item.html
■             cart-item.spec.ts
■             cart-item.ts
■           cart-summary/
■             cart-summary.css
■             cart-summary.html
■             cart-summary.spec.ts
■             cart-summary.ts
■         pages/
■           cart/
■             cart.css
■             cart.html
■             cart.spec.ts
■             cart.ts
■       checkout/
■         pages/
■           checkout/
■             checkout.css
■             checkout.html
■             checkout.spec.ts
■             checkout.ts
■       home/
■         components/
■           banner/
■             banner.css
■             banner.html
■             banner.spec.ts
■             banner.ts

```

```

    ■■■ product-card/
    ■■■ product-card.css
    ■■■ product-card.html
    ■■■ product-card.spec.ts
    ■■■ product-card.ts
    ■■■
    ■■■ product-grid/
    ■■■ product-grid.css
    ■■■ product-grid.html
    ■■■ product-grid.spec.ts
    ■■■ product-grid.ts
    ■■■
    ■■■ pages/
    ■■■ home/
    ■■■ home.css
    ■■■ home.html
    ■■■ home.spec.ts
    ■■■ home.ts
    ■■■
    ■■■ products/
    ■■■ pages/
    ■■■ product-details/
    ■■■ product-details.css
    ■■■ product-details.html
    ■■■ product-details.spec.ts
    ■■■ product-details.ts
    ■■■
    ■■■ layouts/
    ■■■ footer/
    ■■■ footer.css
    ■■■ footer.html
    ■■■ footer.spec.ts
    ■■■ footer.ts
    ■■■
    ■■■ header/
    ■■■ header.css
    ■■■ header.html
    ■■■ header.spec.ts
    ■■■ header.ts
    ■■■
    ■■■ models/
    ■■■ cart-item.ts
    ■■■ order.ts
    ■■■ product.ts
    ■■■ user.ts
    ■■■
    ■■■ shared/
    ■■■ components/
    ■■■
    ■■■ hero-slider/
    ■■■ hero-slider.css
    ■■■ hero-slider.html
    ■■■ hero-slider.spec.ts
    ■■■ hero-slider.ts
    ■■■
    ■■■ shopping-cards/
    ■■■ shopping-cards.css
    ■■■ shopping-cards.html
    ■■■ shopping-cards.spec.ts
    ■■■ shopping-cards.ts
    ■■■
    ■■■ app.config.ts
    ■■■ app.css
    ■■■ app.html
    ■■■ app.routes.ts
    ■■■ app.spec.ts
    ■■■ app.ts
    ■■■
    ■■■ index.html
    ■■■ main.ts
    ■■■ material-theme.scss
    ■■■ styles.css
    ■■■
    ■■■ angular.json
    ■■■ db.json
    ■■■ package.json
    ■■■ package-lock.json
    ■■■ tsconfig.app.json
    ■■■ tsconfig.json

```

Folder Responsibility Matrix

Folder	Responsibility
`core/`	Application-wide singleton infrastructure
`core/api/`	API endpoint configuration
`core/guards/`	Navigation authorization
`core/services/`	Business logic, HTTP access, shared state
`features/`	Domain-specific application capabilities

`layouts/`	Global structural UI	
`models/`	TypeScript domain models	
`shared/`	Reusable application-independent components	
`public/assets/`	Static files	
`db.json`	Development REST API database	

2. Core Concepts & Angular Features Deep-Dive

2.1 Standalone Components

What They Are

Standalone components remove the requirement for Angular NgModule declarations.

A standalone component imports its own dependencies.

Why They Were Used

The application uses Angular 22.

Standalone architecture provides:

- explicit dependencies;
- simpler application bootstrap;
- easier feature isolation;
- reduced NgModule boilerplate;
- better compatibility with modern lazy loading.

Exact Project Example

product-grid.ts:

```
import { Component, input } from '@angular/core';

import { Product } from '../../../models/product';
import { ProductCard } from '../product-card/product-card';

@Component({
  selector: 'app-product-grid',
  standalone: true,
  imports: [
    ProductCard
  ],
  templateUrl: './product-grid.html',
  styleUrls: ['./product-grid.css']
})
export class ProductGrid {

  products = input.required<Product[]>();
}
```

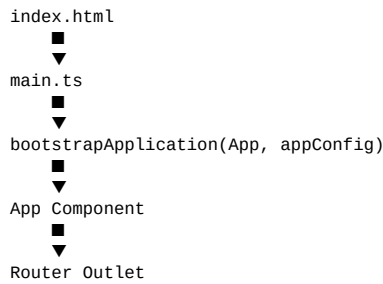
The component imports ProductCard directly.

The data dependency is declared using Angular's signal-based input API:

```
products = input.required<Product[]>();
```

2.2 Application Bootstrap

The modern standalone bootstrap chain is:



Conceptually, main.ts is structured as:

```
import { bootstrapApplication } from '@angular/platform-browser';

import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch(error => console.error(error));
```

The application-level providers belong in app.config.ts.

```
import {
  ApplicationConfig,
  provideBrowserGlobalErrorListeners,
  provideZonelessChangeDetection
} from '@angular/core';

import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(),
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

The important architectural principle is:

```
NgModule providers
  ↓ replaced by
ApplicationConfig providers
```

2.3 Dependency Injection and Services

What Dependency Injection Is

Dependency Injection allows Angular to create and supply class dependencies automatically.

Instead of:

```
const productService = new ProductService();
```

the component requests the service:

```
private productService = inject(ProductService);
```

Angular's dependency injector manages service creation.

Why Services Are Used

The application separates UI concerns from business logic.

Components should handle:

```
Template rendering
User interaction
Navigation triggers
Presentation state
```

Services handle:

```
HTTP communication
Authentication
Cart state
Product retrieval
Cross-component state
```

Singleton Service Architecture

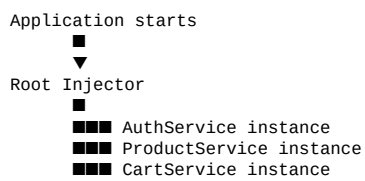
The primary services are:

```
AuthService
ProductService
CartService
```

They use application-level singleton scope:

```
@Injectable({
  providedIn: 'root'
})
```

This means:



Every component injecting CartService receives the same service instance.

That is critical because:

```
ProductCard
Header
Cart
Checkout
```

must all observe the same cart state.

2.4 Product Service

Responsibility

ProductService owns product API communication.

Its conceptual API contract is:

```
getProducts(): Observable<Product[]>
getProduct(id: string): Observable<Product>
```

The implementation follows:

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { Product } from '../../models/product';

@Injectable({
```

```

    providedIn: 'root'
  })
  export class ProductService {

    private http = inject(HttpClient);

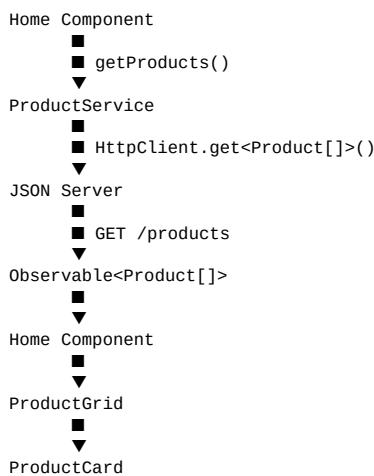
    private apiUrl = 'http://localhost:3000/products';

    getProducts(): Observable<Product[]> {
      return this.http.get<Product[]>(this.apiUrl);
    }

    getProduct(id: string): Observable<Product> {
      return this.http.get<Product>(
        `${this.apiUrl}/${id}`
      );
    }
  }
}

```

Product Collection Pipeline



2.5 Directives and Pipes

Current Implementation

The application does not define custom directives or custom pipes.

Angular's built-in template capabilities are used instead.

Examples include:

```

[routerLink]="['/products', item.id]"
[src]="item.product.image"
[alt]="item.product.title"
[disabled]="placingOrder"
[formGroup]="addressForm"
formControlName="fullName"

```

The application also uses Angular 22 control-flow syntax:

```

@if (placingOrder) {
  Placing your order...
} @else {
  Place your order
}

```

and:


```

    @for (product of products(); track product.id) {

        <app-product-card
            [product]="product">
        </app-product-card>

    }

```

This replaces older patterns such as:

```
*ngIf
```

and:

```
*ngFor
```

for the corresponding templates.

2.6 Routing and Navigation

Router Architecture

The router connects URL state with feature pages.

The primary navigation graph is:

```

/
■
■■■ /login
■
■■■ /register
■
■■■ /products/:id
■
■■■ /cart
■
■■■ /checkout
■
■■■ /order-success

```

The dynamic product route is:

```

{
  path: 'products/:id',
  loadComponent: () =>
    import(
      './features/products/pages/product-details/product-details'
    ).then(component => component.ProductDetails)
}

```

The `:id` portion is a dynamic route parameter.

Example:

```

/products/1
/products/2
/products/8

```

Lazy Loading

Modern standalone components can be lazy loaded using:

```
loadComponent()
```

instead of eager imports.

Example:

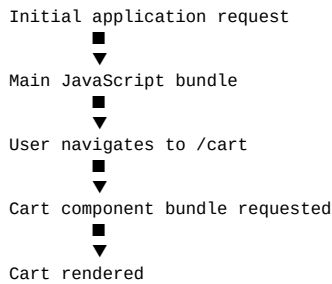
```

{
  path: 'cart',
  loadComponent: () =>

```

```
import(
  './features/cart/pages/cart/cart'
).then(component => component.Cart)
}
```

The result is:



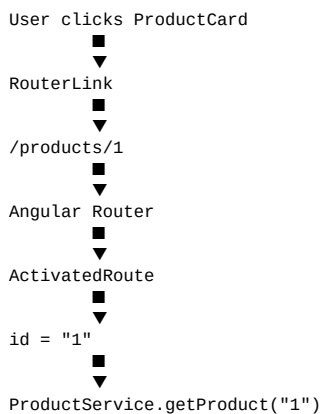
This reduces the amount of code required during initial application startup.

Dynamic Route Parameters

The product-details page retrieves the product identifier from the route.

```
private route = inject(ActivatedRoute);
```

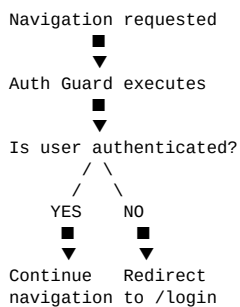
The parameter pipeline is:



Authentication Guard

The guard protects authenticated routes.

The logic is:



This protects pages such as checkout from unauthorized navigation.

Guards Not Implemented

The project currently does not require:

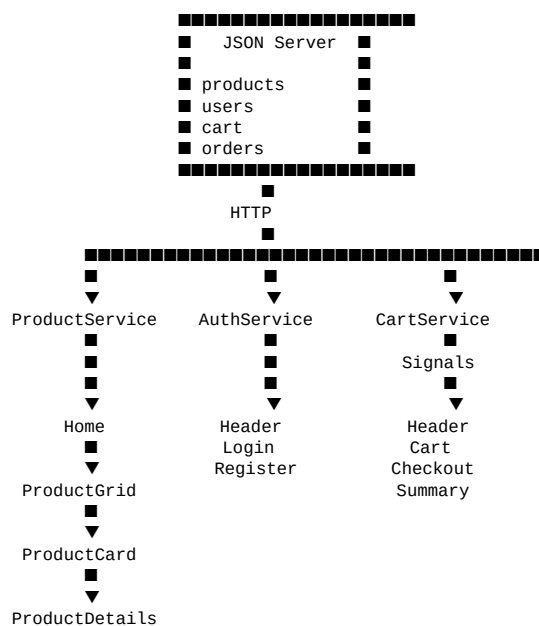
- CanDeactivate/Leave Guard;
- Resolver;
- Role Guard.

Therefore, those mechanisms should not be documented as existing features.

3. Data Transfer Passage & State Management

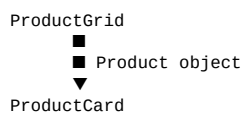
3.1 Complete Application Data Architecture

The most important architectural diagram is:



3.2 Parent-to-Child Communication

The clearest example is:



product-grid.ts:

```
products = input.required<Product[]>();
```

product-grid.html:

```
<div class="product-grid">
  @for (product of products(); track product.id) {
    <app-product-card
      [product]="product">
    </app-product-card>
  }
</div>
```

Mechanically:

```
products()  
  ▼  
@for iterates collection  
  ▼  
product variable created  
  ▼  
[product]="product"  
  ▼  
ProductCard input updated  
  ▼  
ProductCard template re-renders
```

3.3 Child-to-Parent Communication

Angular's standard child-to-parent mechanism is:

```
output()
```

or historically:

```
@Output()  
EventEmitter
```

However, the final project architecture primarily avoids unnecessary event bubbling.

For example, ProductCard does not need:

```
ProductCard  
  ▼  
ProductGrid  
  ▼  
Home  
  ▼  
CartService
```

Instead, it communicates directly with the shared singleton service:

```
ProductCard  
  ▼  
CartService
```

This reduces event propagation through components that do not own the cart business logic.

3.4 Sibling and Unrelated Component Communication

This is one of the most important patterns in the project.

Consider:

```
ProductCard  
Header  
Cart Page  
Checkout Page
```

These components are unrelated in the component tree.

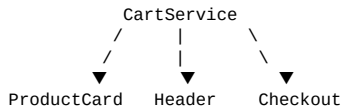
Passing data through @Input() would require:

```
ProductCard  
  ▼  
ProductGrid  
  ▼
```



That would create prop drilling.

Instead:



The shared service acts as the communication channel.

3.5 Signal-Based State Management

The application does not use:

- NgRx;
- Akita;
- Redux.

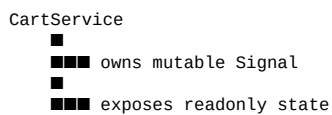
The application uses Angular Signals in singleton services.

The conceptual cart implementation is:

```

private cartItems = signal<CartItem[]>([]);
readonly cart = this.cartItems.asReadonly();
  
```

State ownership is:



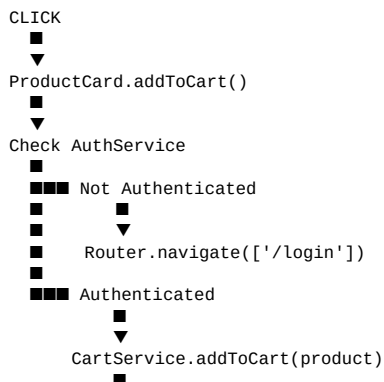
Components consume:

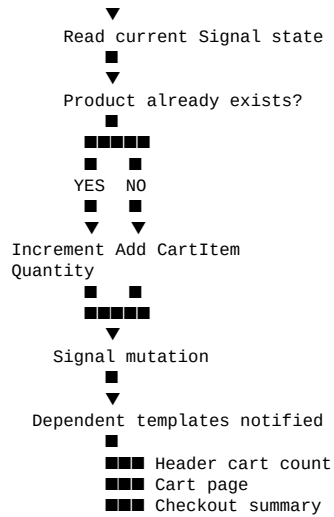
```

cartService.cart()
  
```

Cart Mutation Pipeline

When the user presses Add to Cart:



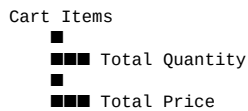


This is the core state-management mechanism.

3.6 Computed State

Derived state should not be independently stored.

For example:



These values are derived.

Conceptually:

```

totalItems = computed(() =>
  this.cartItems().reduce(
    (total, item) => total + item.quantity,
    0
  )
);

```

and:

```

totalPrice = computed(() =>
  this.cartItems().reduce(
    (total, item) =>
      total + item.product.price * item.quantity,
    0
  )
);

```

The checkout template consumes:

```

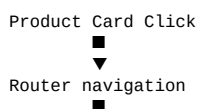
■{{ cartService.totalPrice() }}

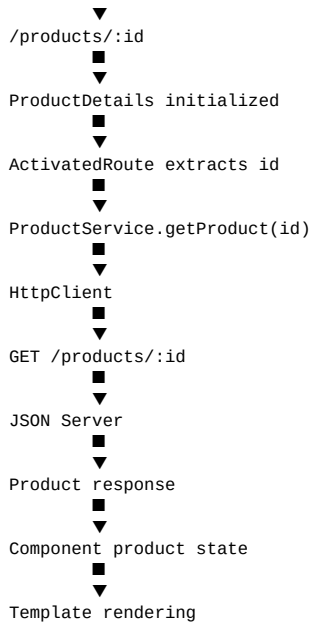
```

The order summary therefore always derives its value from current cart state.

3.7 Product Retrieval Pipeline

The product-details flow is:





This is why the earlier debugging result:

```

API Response: Object
Product Assigned: Object
  
```

was important.

It proved:

```

Backend           WORKING
HTTP request      WORKING
API response      WORKING
  
```

and narrowed the original issue to component/template state handling.

3.8 Related Product Pipeline

The related-product implementation follows:

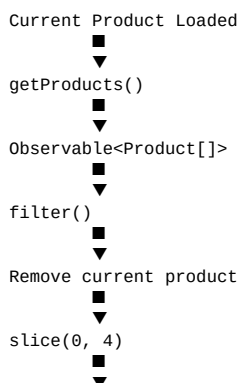
```

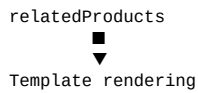
this.productService
  .getProducts()
  .subscribe(products => {

    this.relatedProducts = products
      .filter(
        product =>
          product.id !== this.product?.id
      )
      .slice(0, 4);

  });
  
```

Mechanically:





3.9 RxJS Usage

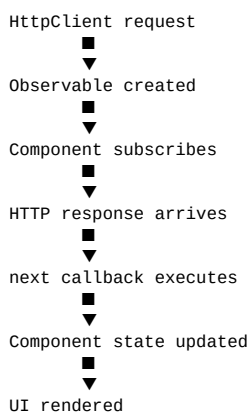
The project uses RxJS primarily because HttpClient returns Observables.

Example:

```

this.productService
  .getProducts()
  .subscribe(products => {
    this.relatedProducts = products;
  });
  
```

The flow is:



Operators Actually Required by the Current Architecture

The current implementation relies primarily on:

- Observable creation by HttpClient;
- subscribe() consumption;
- JavaScript array filter();
- JavaScript array slice().

Operators such as:

```

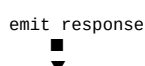
switchMap
mergeMap
shareReplay
  
```

were not established as part of the final implementation and therefore should not be falsely documented as currently used.

A future refactoring could replace nested imperative subscriptions with switchMap, but that is an optimization opportunity rather than existing architecture.

3.10 Subscription Management

For Angular HttpClient, requests generally:




```
complete
```

Therefore, simple HTTP subscriptions do not remain active indefinitely.

The application does not currently establish a universal:

```
takeUntilDestroyed()
```

strategy.

For long-lived streams, such a strategy would be necessary.

Example production pattern:

```
private destroyRef = inject(DestroyRef);

someLongLivedObservable$
  .pipe(
    takeUntilDestroyed(this.destroyRef)
  )
  .subscribe(value => {
    // consume value
  });
```

But this should be understood as a recommended enhancement, not falsely described as existing project code.

4. API Integration & Network Layer

4.1 Backend Architecture

The development backend uses json-server.

The database structure is:

```
{
  "products": [],
  "users": [],
  "cart": [],
  "orders": []
}
```

The REST resources are automatically generated.

```
GET    /products
GET    /products/:id
POST   /products
PUT    /products/:id
PATCH /products/:id
DELETE /products/:id

GET    /users
POST   /users

GET    /cart
POST   /cart
PATCH /cart/:id
DELETE /cart/:id

GET    /orders
POST   /orders
```

4.2 HTTP Client

Angular provides HttpClient through:

```
provideHttpClient()
```

The service injects it:

```
private http = inject(HttpClient);
```

A typed request is:

```
getProducts(): Observable<Product[]> {  
    return this.http.get<Product[]>(  
        this.apiUrl  
    );  
}
```

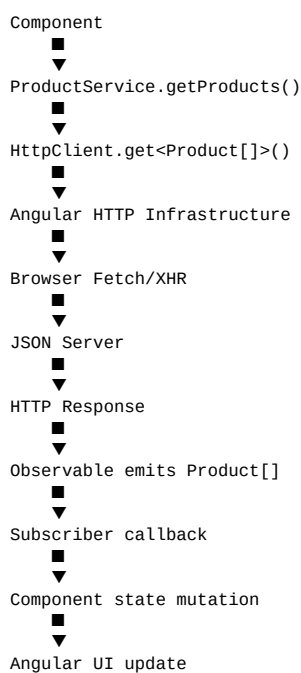
The generic:

```
<Product[]>
```

provides compile-time typing.

4.3 Request Lifecycle

A product request moves through:



4.4 Interceptors

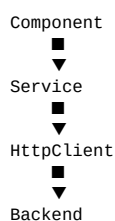
The directory:

```
core/interceptors/
```

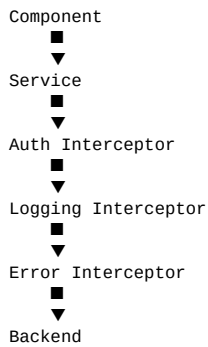
exists as an architectural boundary.

However, the final project does not establish a production authentication-token interceptor or global error interceptor.

Therefore, the current pipeline is effectively:



not:



This distinction matters in accurate technical documentation.

4.5 Data Models and Type Safety

Product

The product domain contains:

```
export interface Product {  
  id: string;  
  title: string;  
  desc: string;  
  price: number;  
  image: string;  
  ratings: number;  
}
```

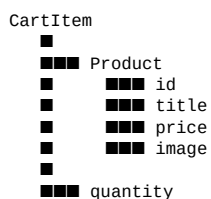
The matching JSON shape is:

```
{  
  "id": "1",  
  "title": "xbox one x 1TB",  
  "desc": "Microsoft Xbox One X 1Tb Console With Wireless Controller",  
  "price": 45000,  
  "image": "https://example.com/product.jpg",  
  "ratings": 4  
}
```

Cart Item

```
import { Product } from './product';  
  
export interface CartItem {  
  product: Product;  
  quantity: number;  
}
```

The relationship is:



User

The user domain follows:

```
export interface User {  
  id?: string;  
  name: string;  
  email: string;  
  password: string;  
}
```

Order

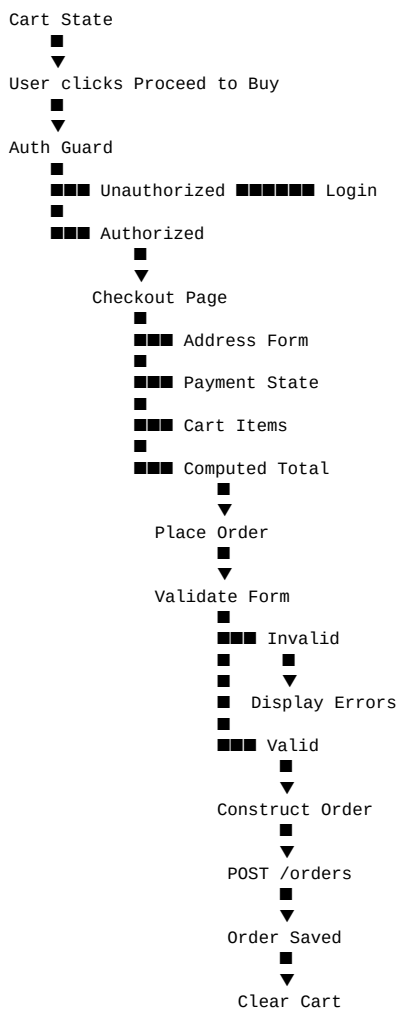
The order domain logically combines:

```
Order  
■  
■■■ User  
■■■ CartItems  
■■■ Address  
■■■ Payment Method  
■■■ Total  
■■■ Status
```

This allows the checkout workflow to transform transient cart state into persistent order data.

4.6 Checkout Data Flow

The checkout process is the most complex multi-domain operation.



4.7 Reactive Forms

The checkout page uses Reactive Forms.

Template:

```
<form [formGroup]="addressForm">
```

Controls:

```
<input
  type="text"
  formControlName="fullName">
```

Validation:

```
@if (
  addressForm.controls.fullName.touched &&
  addressForm.controls.fullName.invalid
) {

  <small class="validation-error">
    Please enter your full name.
  </small>

}
```

The state machine is:

```
FormControl
  ■
  ■■■ value
  ■■■ valid / invalid
  ■■■ touched / untouched
  ■■■ dirty / pristine
  ■■■ errors
```

The UI renders validation based on:

```
touched AND invalid
```

This prevents validation errors from displaying before the user interacts with the field.

5. Advanced Optimization & Performance Mechanics

5.1 Change Detection Strategy

The project does not explicitly establish:

```
changeDetection: ChangeDetectionStrategy.OnPush
```

on the feature components.

Therefore, it should not be claimed that OnPush is currently used.

The architecture instead benefits from Angular Signals because Angular can track signal consumers and update dependent views reactively.

Example:

```
cartService.cart()
```

or:

```
cartService.totalPrice()
```

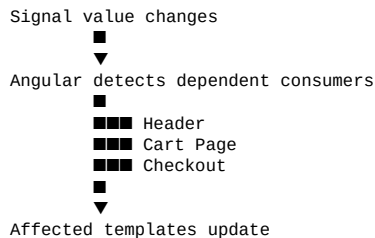
creates explicit reactive dependencies.

5.2 Signal Update Mechanics

Suppose:

```
cartItems.set(updatedItems);
```

The update pipeline is:



This avoids manually synchronizing multiple components.

5.3 List Rendering Optimization

The application uses tracked loops:

```
@for (
  product of products();
  track product.id
) {

  <app-product-card
    [product]="product">
  </app-product-card>

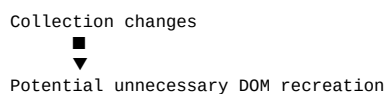
}
```

Tracking by:

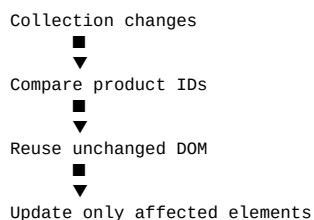
```
product.id
```

allows Angular to associate DOM nodes with stable product identities.

Without stable tracking:



With tracking:



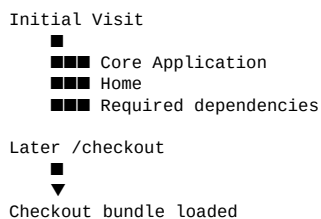
5.4 Lazy Loading

Using:

```
loadComponent()
```

creates page-level code splitting.

The application does not need to load checkout code when the user initially visits the home page.



This improves initial bundle efficiency.

5.5 Security Controls

XSS Prevention

Angular interpolations:

```
{{ product.title }}
```

are escaped by Angular.

Data is not directly injected as arbitrary HTML.

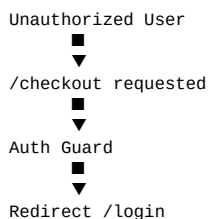
The project avoids patterns such as:

```
<div [innerHTML]="untrustedUserData"></div>
```

which reduces XSS exposure.

Route Protection

Sensitive workflows such as checkout are protected through an authentication guard.



However, an important production principle is:

Client-side route guards are UX controls, not true backend authorization.

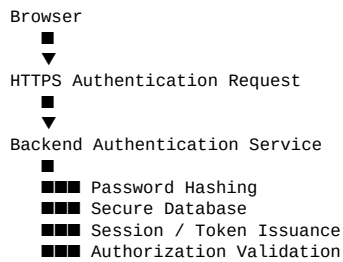
A production backend must independently verify authentication and authorization.

Authentication Storage

The current JSON Server authentication implementation is suitable for learning and portfolio demonstration, but not production authentication security.

The current model contains user passwords in JSON data.

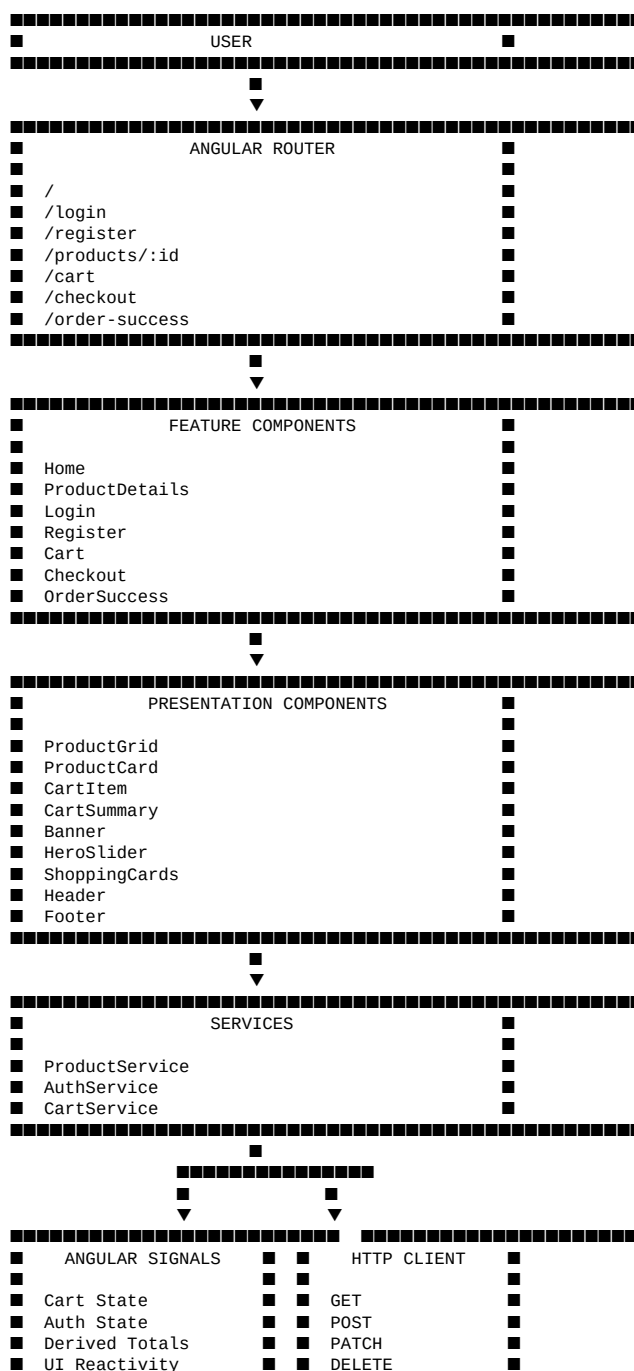
A real production implementation must replace this with:

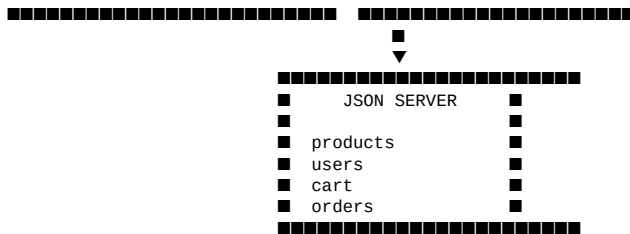


Passwords should never be stored in plaintext.

6. Complete End-to-End Application Flow

The complete application architecture can finally be represented as:



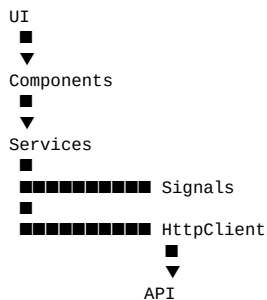


7. Final Architectural Assessment

The project demonstrates a modern Angular architecture centered on:

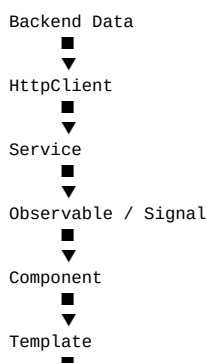
- Angular 22
- +
- Standalone Components
- +
- Feature-Based Organization
- +
- Dependency Injection
- +
- Singleton Services
- +
- Angular Signals
- +
- Computed State
- +
- RxJS HTTP Streams
- +
- HttpClient
- +
- Reactive Forms
- +
- Route Parameters
- +
- Authentication Guards
- +
- Lazy Loading
- +
- Typed Domain Models
- +
- JSON Server REST API

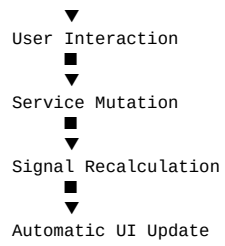
The strongest architectural decision is the separation:



This separation prevents the Angular components from becoming responsible for API communication, global state ownership, and presentation simultaneously.

The central application data path is:





That is the core technical architecture of the Amazon Clone application we built.